UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información
y Ciencias de la Computación

Curso: Seguridad y Auditoría de Sistemas

PROYECTO FINAL

Sistema de autenticación multifactor (MFA) para aplicaciones web: «Ceiba»

Catedrático: Ing. _____

Sección: _____ Jornada: _____

Integrantes del grupo

Nombre completo	Carné
Daniel Alejandro González Juárez	0910-22-4469
Alex Ricardo Pereira	0910-21-14444

Guatemala, octubre de 2026

Índice

Informe ejecutivo..... **3**
Justificación..... **5**
Contenido..... **6**
Objetivos..... 6
Resultados esperados..... 6
Alcances y limitaciones..... 7
Propuesta de solución..... 8
Pruebas de resistencia frente a ataques..... 13
Seguridad, usabilidad y eficiencia..... 15
Cuadro de porcentaje de participación..... 16
Conclusiones..... **17**
Recomendaciones..... **17**
Referencias..... **17**
Anexos..... **19**

Informe ejecutivo

Para: autoridades del curso y partes interesadas en la seguridad de las aplicaciones web de la organización.

Asunto: diseño, implementación y validación de un sistema de autenticación multifactor (MFA).

La mayoría de aplicaciones web todavía confían en una sola barrera: el usuario y la contraseña. Esa barrera falla con frecuencia, porque las contraseñas se reutilizan, se filtran en brechas de terceros y se roban mediante phishing. Según el Data Breach Investigations Report 2025 de Verizon, el abuso de credenciales fue el vector de acceso inicial más común en las brechas analizadas, y el elemento humano estuvo presente en alrededor del 60 % de ellas.

Para responder a ese riesgo, el grupo diseñó e implementó **Ceiba**, una aplicación web funcional en la que iniciar sesión requiere dos factores independientes: algo que el usuario sabe (su contraseña) y algo que tiene (su teléfono o su buzón de correo). El sistema ofrece tres métodos de segundo factor: **app de autenticación TOTP** (Google Authenticator, Microsoft Authenticator, Authy), **código por correo electrónico** y **código por SMS**, además de **códigos de respaldo** de un solo uso para emergencias.

El sistema no se limita a pedir un código. Incorpora controles que lo hacen resistente a los ataques que buscan saltarse el segundo factor: bloqueo progresivo ante fuerza bruta, anulación de la verificación tras cinco códigos incorrectos, códigos que vencen a los cinco minutos y no pueden reutilizarse, cifrado de los secretos con AES-256-GCM, contraseñas con bcrypt, protección contra CSRF, fijación de sesión, clickjacking y enumeración de usuarios, y una bitácora de auditoría con panel de monitoreo para el administrador.

Resultados principales

- **15 de 15 ataques resistidos** en pruebas automatizadas ejecutadas por HTTP contra el sistema, entre ellos robo de contraseña, fuerza bruta del código, reutilización de códigos, inyección SQL, CSRF y robo de la base de datos.
- **Contraseña robada no equivale a cuenta comprometida:** en la prueba A2 el atacante conocía la contraseña correcta y aun así no obtuvo acceso a ningún dato.
- **Probabilidad de adivinar el código de 0.0005 % por intento de acceso**, porque solo se permiten cinco intentos antes de exigir de nuevo la contraseña.
- **Usabilidad:** el flujo completo toma menos de 30 segundos; el código se envía al escribir el sexto dígito y el usuario puede marcar un dispositivo de confianza por 30 días.
- **Eficiencia:** la validación de la contraseña tarda cerca de 0.3 s (costo intencional de bcrypt) y la del código menos de un milisegundo; no requiere servidores de base de datos adicionales.

Recomendación

Se recomienda adoptar la autenticación multifactor como requisito obligatorio en las aplicaciones web que manejen información sensible, priorizando la app de autenticación como método principal y dejando el SMS como respaldo. El sistema entregado puede usarse como base: está documentado, se instala con dos comandos y sus pruebas de ataque pueden repetirse en cualquier momento con npm run

test:ataques. Microsoft Research reportó en 2023 que la MFA reduce en 99.22 % el riesgo de compromiso de cuentas, y en 98.56 % incluso cuando las credenciales ya se habían filtrado.

Justificación

La autenticación es la puerta de entrada de cualquier aplicación. Si falla, el resto de controles (permisos, cifrado, respaldos) protegen datos a los que el atacante ya tiene acceso legítimo. Por eso la forma en que una aplicación confirma la identidad de sus usuarios es una de las decisiones de seguridad más importantes de su diseño.

El esquema tradicional de usuario y contraseña presenta debilidades estructurales que no se resuelven solo con exigir contraseñas más largas:

- **Phishing:** un correo o sitio falso convence al usuario de escribir su contraseña, que el atacante usa de inmediato.
- **Reutilización y relleno de credenciales:** las personas repiten contraseñas entre servicios; una filtración en un sitio abre la puerta de otros.
- **Fuerza bruta y diccionario:** programas automatizados prueban miles de contraseñas comunes por minuto.
- **Robo de bases de datos:** si las contraseñas se guardan sin un algoritmo adecuado, una sola brecha las expone todas.

La autenticación multifactor ataca el problema de raíz: aunque el atacante obtenga la contraseña, le falta un segundo elemento que solo el usuario legítimo posee. La guía NIST SP 800-63B, referencia internacional para identidad digital, establece que los niveles de garantía AAL2 y superiores requieren dos factores distintos, y el estándar OWASP ASVS incluye la MFA entre los controles de verificación de aplicaciones.

Para el contexto de las organizaciones guatemaltecas, donde los servicios en línea, la banca electrónica y los sistemas internos han crecido de forma acelerada, contar con un modelo de MFA propio, entendible y verificable permite:

- Reducir el riesgo de accesos no autorizados derivados de contraseñas filtradas.
- Cumplir con buenas prácticas y marcos de referencia exigidos en auditorías de sistemas.
- Detectar intentos de intrusión gracias a la bitácora de eventos de seguridad.
- Capacitar a usuarios y equipos técnicos con una solución que se puede inspeccionar y probar.

El proyecto se justifica, además, desde la perspectiva académica del curso: obliga a llevar a la práctica conceptos de criptografía, gestión de sesiones, control de acceso, auditoría y pruebas de penetración, y a demostrar con evidencia, no solo con teoría, que el sistema resiste los ataques para los que fue diseñado.

Contenido

Objetivos

Objetivo general

Diseñar e implementar un sistema de autenticación multifactor para aplicaciones web que combine la contraseña con un segundo factor de verificación (aplicación de autenticación, correo electrónico o SMS), y comprobar mediante pruebas controladas que es seguro, usable, eficiente y resistente a ataques como el robo de contraseñas.

Objetivos específicos

1. Integrar en el proceso de inicio de sesión tres métodos de segundo factor: TOTP según el RFC 6238, código de un solo uso por correo y código por SMS, con códigos de respaldo para recuperar el acceso.
2. Proteger las credenciales y secretos almacenados mediante algoritmos reconocidos: bcrypt para contraseñas, AES-256-GCM para secretos TOTP y HMAC-SHA256 para códigos de un solo uso.
3. Implementar controles contra ataques de fuerza bruta, reutilización de códigos, CSRF, fijación de sesión, clickjacking, inyección SQL y enumeración de usuarios.
4. Diseñar una interfaz clara, accesible y adaptable a dispositivos móviles que guíe al usuario en cada paso sin sacrificar seguridad.
5. Registrar los eventos de seguridad en una bitácora de auditoría y ofrecer un panel de monitoreo para el administrador.
6. Validar la resistencia del sistema con un conjunto automatizado y repetible de pruebas de ataque, documentando la evidencia obtenida.

Resultados esperados

Al finalizar el proyecto se espera contar con los siguientes productos y resultados verificables:

Tabla 1. *Resultados esperados y forma de verificación*

Resultado esperado	Indicador de cumplimiento	Estado
Aplicación web con inicio de sesión en dos pasos	Ningún usuario accede a su cuenta sin validar el segundo factor	Cumplido
Tres métodos de segundo factor y códigos de respaldo	Alta, uso y eliminación de TOTP, correo y SMS desde la cuenta	Cumplido
Resistencia al robo de contraseñas	Un atacante con la contraseña correcta no obtiene acceso (prueba A2)	Cumplido
Resistencia a fuerza bruta	Bloqueo tras 5 contraseñas o 5 códigos incorrectos (pruebas A1, A3, A15)	Cumplido
Almacenamiento seguro	Ningún secreto en texto plano en la base de datos (prueba A13)	Cumplido

Resultado esperado	Indicador de cumplimiento	Estado
Trazabilidad	Cada evento de autenticación queda en la bitácora con fecha, IP y nivel	Cumplido
Usabilidad	Flujo completo en menos de 30 s, interfaz adaptable a móvil, mensajes claros	Cumplido
Evidencia de pruebas	Reporte automático con 15 ataques documentados	15/15 resistidos

Alcances y limitaciones

Alcances

- Registro de usuarios con política de contraseñas robustas y verificación contra contraseñas filtradas (Have I Been Pwned con k-anonimato).
- Inicio de sesión en dos pasos con selección de método principal y métodos de reserva, cambio de método durante la verificación y códigos de respaldo.
- Configuración obligatoria del segundo factor al crear la cuenta: no existen cuentas activas protegidas solo con contraseña.
- Opción de recordar un dispositivo de confianza por 30 días, revocable desde la cuenta y anulada automáticamente al cambiar la contraseña.
- Gestión de la cuenta: métodos de verificación, códigos de respaldo, cambio de contraseña, dispositivos y actividad de seguridad.
- Notificaciones por correo ante inicios de sesión desde otra IP, bloqueos, uso de códigos de respaldo y cambios de seguridad.
- Panel de monitoreo para administradores con indicadores, gráfico por hora, bitácora y desbloqueo de cuentas.
- Conjunto automatizado de 15 pruebas de ataque con reporte en HTML y JSON.

Limitaciones

- El envío real de correos y SMS depende de credenciales de un proveedor (SMTP y Twilio). Para la demostración, sin esas credenciales, los mensajes se muestran en una bandeja de demostración que debe desactivarse en producción.
- Las sesiones se guardan en memoria del servidor; para operar varias instancias se debe usar un almacén compartido como Redis.
- No incluye llaves físicas FIDO2/WebAuthn ni biometría; se plantean como mejora futura.
- La recuperación de cuenta sin ningún factor disponible (sin teléfono ni códigos de respaldo) requiere intervención de un administrador.
- El sistema se probó en un entorno local; una puesta en producción requiere HTTPS, un dominio propio y endurecimiento del servidor.

Propuesta de solución

La propuesta del grupo es **Ceiba**, una aplicación web que implementa la autenticación multifactor de extremo a extremo. El nombre hace referencia al árbol nacional de Guatemala: así como la ceiba crece en capas, la seguridad de la cuenta se construye con capas independientes que un atacante tendría que vencer por separado.

Arquitectura

La solución sigue una arquitectura cliente-servidor de tres capas. El navegador solo recibe HTML, CSS y JavaScript propios servidos por la aplicación, sin bibliotecas externas ni scripts en línea, lo que permite aplicar una política de seguridad de contenido (CSP) estricta. El servidor, construido con Node.js y Express, concentra la lógica de autenticación, la criptografía y el control de acceso. Los datos se guardan en SQLite, un motor embebido que no requiere instalación adicional.

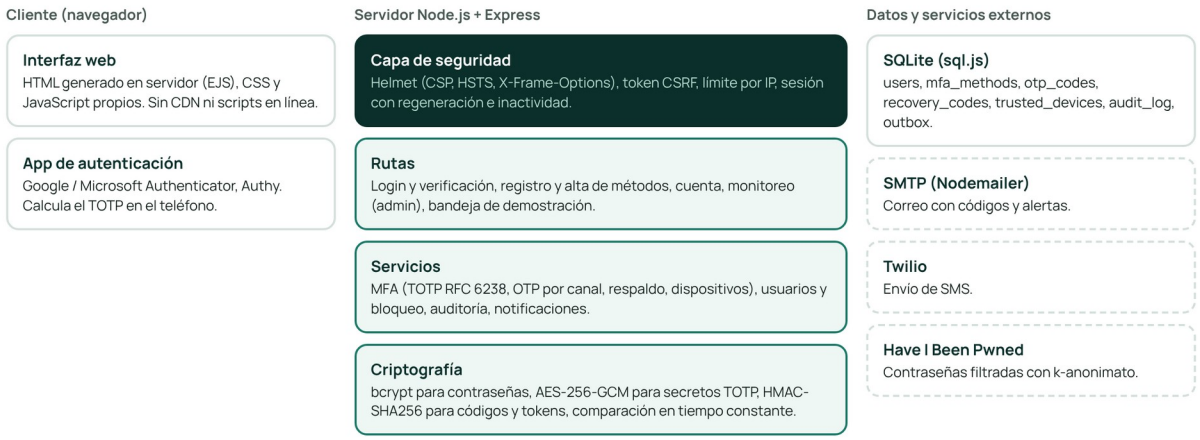


Figura 1. *Arquitectura de la solución Ceiba*

Flujo de autenticación

El inicio de sesión se divide en dos etapas separadas por una sesión pendiente. Validar la contraseña no da acceso a nada: solo habilita, durante diez minutos, la pantalla de verificación. La sesión completa se crea únicamente cuando el segundo factor es correcto, y en cada cambio de nivel se genera un identificador de sesión nuevo.

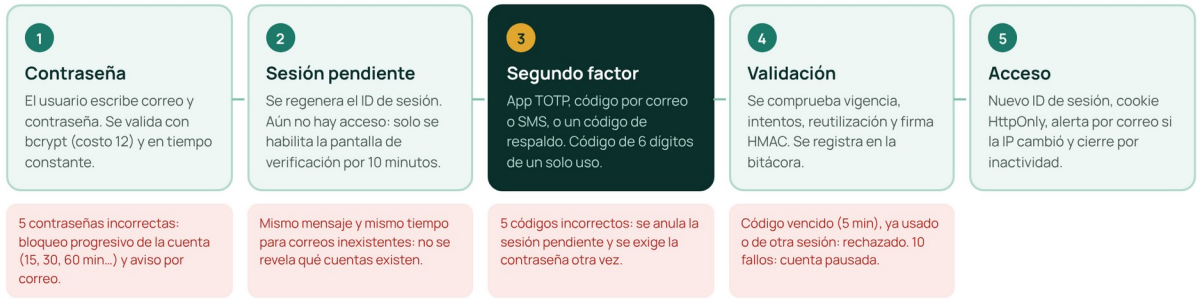


Figura 2. *Flujo de autenticación en dos pasos y respuesta ante fallos*

Métodos de segundo factor

Tabla 2. *Comparación de los factores implementados*

Método	Cómo funciona	Fortalezas	Riesgos y mitigación
App de autenticación (TOTP)	Secreto de 160 bits compartido mediante QR. El teléfono calcula un código de 6 dígitos cada 30 s (RFC 6238).	Funciona sin señal; el código no viaja por la red; no depende de terceros.	Pérdida del teléfono: códigos de respaldo. Reutilización: cada paso se acepta una sola vez.
Correo electrónico	Código aleatorio de 6 dígitos enviado por SMTP; vence en 5 min.	Todos los usuarios tienen correo; no requiere instalar nada.	Buzón comprometido: se recomienda combinar con TOTP. Código guardado como HMAC.
SMS	Código de 6 dígitos enviado por la API de Twilio; vence en 5 min.	Cómodo y conocido por usuarios no técnicos.	Duplicado de SIM: se presenta como método de respaldo, no principal.
Códigos de respaldo	10 códigos XXXX-XXXX de un solo uso generados al activar la MFA.	Permiten entrar si se pierde el método principal.	Se muestran una sola vez, se guardan como HMAC y su uso genera alerta.

Controles de seguridad implementados

Tabla 3. *Controles de seguridad y amenaza que mitigan*

Control	Implementación	Amenaza mitigada
Almacenamiento de contraseñas	bcrypt con costo 12 (salt individual)	Robo de la base de datos, tablas arcoíris
Política de contraseñas	10+ caracteres, mayúsculas, minúsculas, número y símbolo; rechazo de contraseñas comunes, filtradas o con datos personales	Diccionario, relleno de credenciales
Cifrado de secretos TOTP	AES-256-GCM con vector aleatorio y etiqueta de integridad	Robo de la base de datos
Códigos de un solo uso	Generador criptográfico; guardados como HMAC-SHA256; 5 min de vigencia; 5 intentos	Fuerza bruta, intercepción tardía
Anti-replay TOTP	Se registra el último paso de 30 s aceptado	Phishing en tiempo real, observación
Bloqueo progresivo	5 fallos: 15 min; luego 30, 60… hasta 24 h; aviso por correo	Fuerza bruta sobre la contraseña
Límite por IP	20 intentos fallidos por IP cada 15 min (HTTP 429)	Password spraying, bots
Anti-enumeración	Mismo mensaje, mismo bloqueo y bcrypt ficticio para correos inexistentes	Descubrimiento de cuentas válidas

Control	Implementación	Amenaza mitigada
Gestión de sesión	Regeneración del ID al elevar privilegios, cookie HttpOnly y SameSite=Lax, cierre por 30 min de inactividad	Fijación y secuestro de sesión
Token CSRF	Token sincronizado por sesión, comparado en tiempo constante, en todo POST	Falsificación de peticiones
Cabeceras HTTP	CSP estricta, X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy	XSS, clickjacking
Consultas parametrizadas	Todas las consultas SQL usan parámetros enlazados	Inyección SQL
Reautenticación	Eliminar métodos, regenerar códigos o cambiar contraseña exige la contraseña actual	Uso indebido de una sesión abierta
Auditoría	Bitácora con evento, nivel, IP, navegador y fecha; panel de monitoreo	Falta de trazabilidad, detección tardía

Modelo de datos

La base de datos se compone de siete tablas. Ninguna guarda secretos en texto plano:

Tabla 4. *Tablas de la base de datos*

Tabla	Contenido principal
users	Nombre, correo, teléfono, hash bcrypt, rol, intentos fallidos, bloqueo, último acceso
mfa_methods	Tipo de método (totp, email, sms), destino, secreto TOTP cifrado, último paso usado, método principal
otp_codes	HMAC del código, propósito, canal, intentos, vigencia, estado de consumo
recovery_codes	HMAC de cada código de respaldo y fecha de uso
trusted_devices	HMAC del token del dispositivo, navegador, IP y vencimiento
audit_log	Evento de seguridad, nivel, detalle, IP, navegador y fecha
outbox	Mensajes simulados de correo y SMS (solo modo demostración)

Tecnologías utilizadas

Tabla 5. *Herramientas y bibliotecas*

Componente	Tecnología	Uso
Servidor	Node.js 18+ y Express 4	Rutas, middleware y servidor HTTP
Base de datos	SQLite mediante sql.js (WebAssembly)	Persistencia sin instalar un motor externo

Componente	Tecnología	Uso
TOTP y QR	otplib, qrcode	Generación y verificación de códigos, código QR de vinculación
Contraseñas	bcryptjs	Hash con salt y costo configurable
Criptografía	Módulo crypto de Node.js	AES-256-GCM, HMAC-SHA256, números aleatorios seguros
Seguridad HTTP	helmet, express-rate-limit, express-session	Cabeceras, límites por IP, sesiones
Notificaciones	Nodemailer (SMTP), API REST de Twilio	Envío de correos y SMS
Interfaz	EJS, CSS y JavaScript propios	Vistas accesibles, modo oscuro y diseño adaptable

Interfaz de usuario

La interfaz se diseñó para que el usuario entienda en todo momento en qué paso está y qué debe hacer. La pantalla de acceso se divide en dos: a la izquierda, un panel con la identidad visual y un indicador del progreso real del flujo (contraseña, código, acceso); a la derecha, el formulario. La paleta se inspira en el jade y la vegetación guatemalteca, con verdes profundos y un acento dorado reservado para el paso actual y el estado de seguridad.



Figura 3. *Pantalla de inicio de sesión (paso 1: contraseña)*

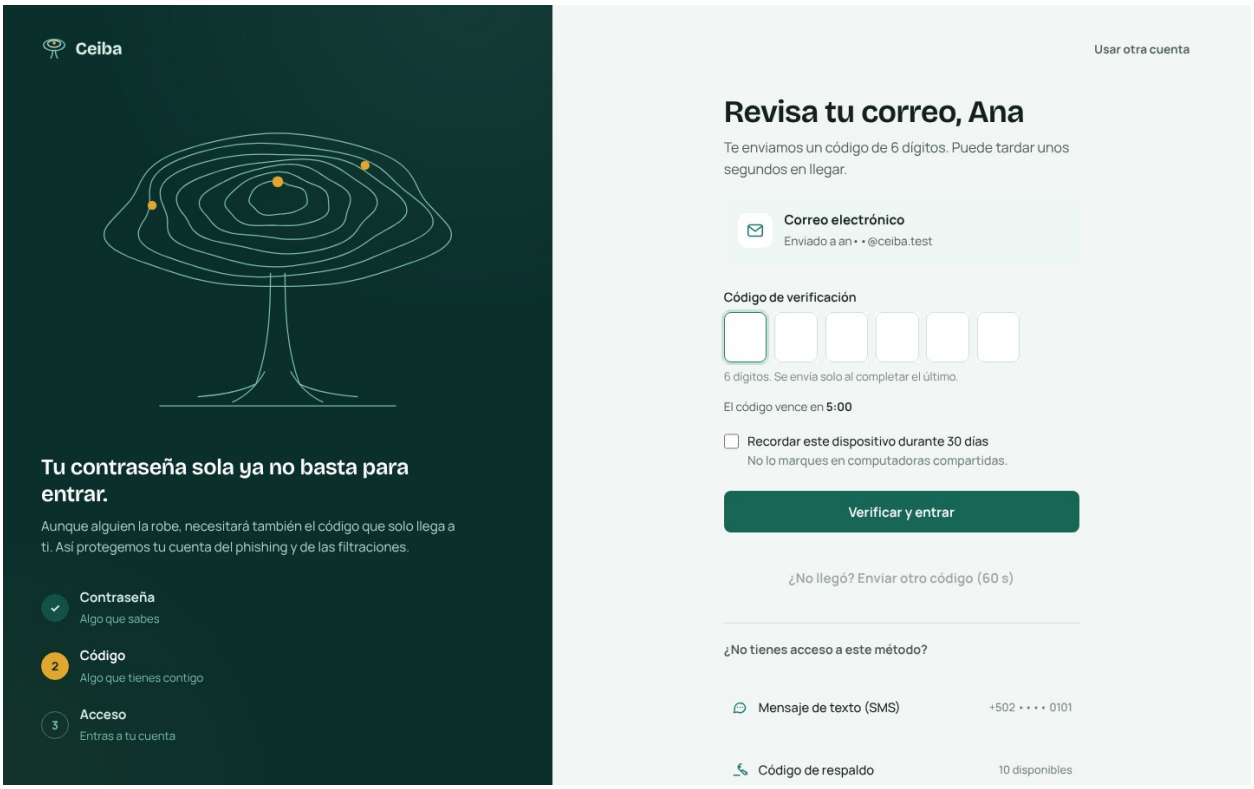


Figura 4. *Verificación con código enviado por correo (paso 2)*

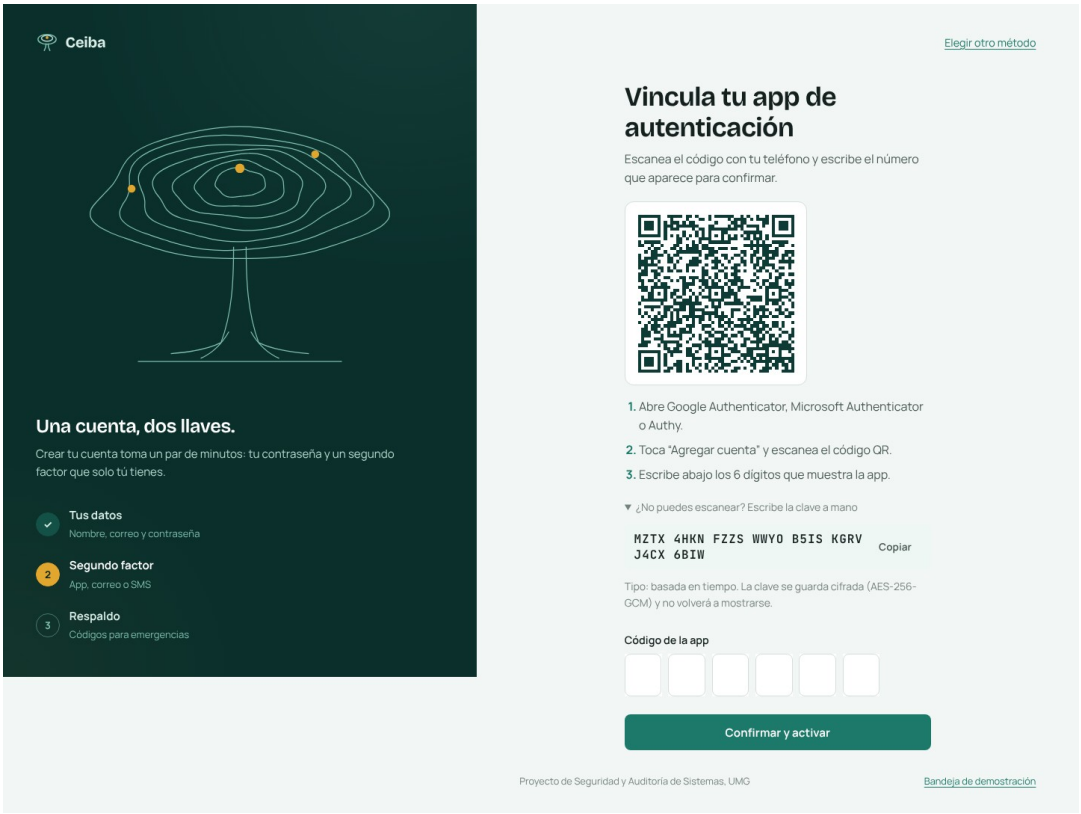


Figura 5. *Vinculación de la app de autenticación mediante código QR*

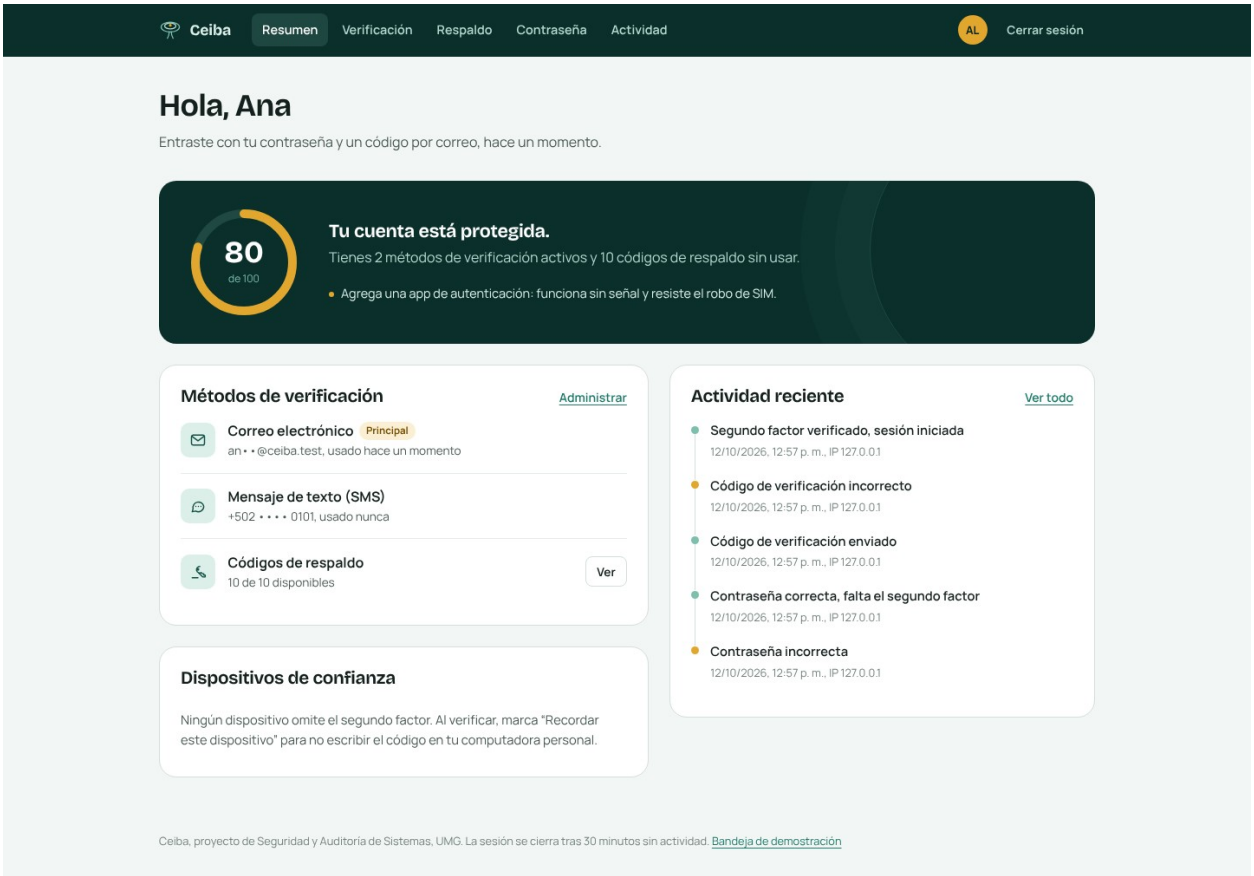


Figura 6. *Panel de la cuenta con el estado de seguridad y la actividad reciente*

Pruebas de resistencia frente a ataques

Para cumplir el requerimiento de probar la resistencia del sistema, el grupo desarrolló un conjunto de pruebas automatizadas (tests/ataques.js). El programa levanta una instancia aislada de la aplicación con una base de datos temporal y ejecuta cada ataque por HTTP, igual que lo haría un atacante real: con su propio navegador simulado, sus propias cookies y sin acceso al servidor.

El escenario central es el **robo de contraseña**: el atacante conoce la contraseña correcta de la víctima, obtenida por phishing o por una filtración, e intenta entrar. Alrededor de ese escenario se probaron los caminos que un atacante usaría para saltarse el segundo factor o comprometer la plataforma.

Tabla 6. *Resultados de las pruebas de ataque (15 de 15 resistidos)*

ID	Ataque	Resultado	Evidencia obtenida
A1	Fuerza bruta con diccionario contra la contraseña	RESISTIÓ	Respuestas: 401, 401, 401, 401, 423, 423, 423. Bloqueo en el intento 5; con la contraseña real también respondió 423 (cuenta pausada 15 min).
A2	Contraseña robada: el atacante la conoce	RESISTIÓ	La contraseña fue aceptada pero el sistema exigió el segundo factor (redirección a /verificar). /panel respondió 302 hacia /verificar; no hubo acceso a datos.

ID	Ataque	Resultado	Evidencia obtenida
A3	Fuerza bruta del código de verificación	RESISTIÓ	Tras 5 códigos incorrectos la sesión se anuló y se volvió a exigir la contraseña. Probabilidad de acierto por sesión: 0.0005 %.
A4	Reutilización de un código de la app (replay)	RESISTIÓ	El usuario legítimo entró con 768066. El mismo código usado por el atacante segundos después fue rechazado: "Ese código ya se usó. Espera a que tu app muestre uno nuevo.".
A5	Reutilización de un código de respaldo	RESISTIÓ	Primer uso de XMHB-E7BJ: acceso concedido. Segundo uso: respuesta 401, "Ese código de respaldo no es válido o ya se usó. Te quedan 4 intentos.".
A6	Uso de un código de correo vencido	RESISTIÓ	Código correcto 774402 enviado 7.5 s después: "Ese código ya venció. Pide uno nuevo con el botón de reenviar.".
A7	Acceso directo a páginas protegidas	RESISTIÓ	/panel → 302 /login; /cuenta/metodos → 302 /login; /cuenta/respaldo → 302 /login; /cuenta/actividad → 302 /login; /admin → 302 /login
A8	Escalada de privilegios a administrador	RESISTIÓ	Sesión válida (/panel 200). /admin respondió 403 y la acción de desbloquear usuarios 403.
A9	Inyección SQL en el formulario de acceso	RESISTIÓ	Respuestas: 400, 401, 400. La tabla users sigue intacta (5 registros). Todas las consultas usan parámetros enlazados.
A10	Falsificación de petición entre sitios (CSRF)	RESISTIÓ	Con sesión activa (/panel 200), las peticiones sin token o con token falso respondieron 403 y 403.
A11	Enumeración de cuentas registradas	RESISTIÓ	Mensajes idénticos: sí ("El correo o la contraseña no coinciden. Revisa ambos e intén..."). Tiempo medio: 363 ms vs 362 ms (diferencia 2 ms).
A12	Fijación de sesión	RESISTIÓ	Identificador antes: s%3Az3vMEi..., tras la contraseña: s%3AeijG8g..., tras el código: s%3ARTKz-r.... Con el identificador plantado, /panel respondió 302 hacia /login.
A13	Robo de la base de datos	RESISTIÓ	Contraseñas con bcrypt costo 12 ($2a$12$DIv6.k...). Secretos TOTP cifrados con AES-256-GCM (JWH7k6N5EIs2x_K0.f...). Códigos de un solo uso y de respaldo guardados como HMAC-SHA256.
A14	Clickjacking, XSS y robo de cookie	RESISTIÓ	CSP: script-src 'self', frame-ancestors 'none'. X-Frame-Options: DENY. X-Content-Type-Options: nosniff. Cookie de sesión HttpOnly y SameSite=Lax.
A15	Ataque masivo desde una misma IP	RESISTIÓ	La IP fue limitada (HTTP 429) tras 14 solicitudes en esta ronda; el límite es de 30 intentos fallidos por IP cada 15 minutos (20 en la configuración normal) y también cuenta los fallos de las pruebas anteriores.

Durante el desarrollo, las pruebas también cumplieron su función de detectar debilidades: una primera ejecución mostró que la cabecera X-Frame-Options se enviaba como SAMEORIGIN y que el límite por IP

contaba también los accesos exitosos, lo que podía bloquear a usuarios legítimos. Ambos puntos se corrigieron (DENY y conteo solo de intentos fallidos) y las pruebas se volvieron a ejecutar hasta obtener el resultado final.



Pruebas de resistencia frente a ataques

Sistema de autenticación multifactor Ceiba. Ejecutado el 12 de octubre de 2026, 12:54 p. m. con Node v22.22.0.

15/15
ataques resistidos

0
vulnerabilidades encontradas

ID	Ataque y objetivo del atacante	Resultado	Evidencia
A1	**Fuerza bruta con diccionario contra la contraseña** Adivinar la contraseña de ana@ceiba.test probando las más comunes	RESISTIÓ	Respuestas: 401, 401, 401, 401, 423, 423, 423. Bloqueo en el intento 5; con la contraseña real también respondió 423 (cuenta pausada 15 min).
A2	**Contraseña robada: el atacante la conoce** Entrar a la cuenta solo con usuario y contraseña correctos	RESISTIÓ	La contraseña fue aceptada pero el sistema exigió el segundo factor (redirección a /verificar). /panel respondió 302 hacia /verificar; no hubo acceso a datos.
A3	**Fuerza bruta del código de verificación** Adivinar el código de 6 dígitos tras robar la contraseña	RESISTIÓ	Tras 5 códigos incorrectos la sesión se anuló y se volvió a exigir la contraseña. Probabilidad de acierto por sesión: 0.0005 %.
A4	**Reutilización de un código de la app (replay)** Usar un código TOTP capturado (phishing en tiempo real u observación)	RESISTIÓ	El usuario legítimo entró con 768066. El mismo código usado por el atacante segundos después fue rechazado: "Ese código ya se usó. Espera a que tu app muestre uno nuevo.".
A5	**Reutilización de un código de respaldo** Usar dos veces un código de respaldo filtrado	RESISTIÓ	Primer uso de XMHB-E7BJ: acceso concedido. Segundo uso: respuesta 401, "Ese código de respaldo no es válido o ya se usó. Te quedan 4 intentos.".

Figura 7. *Reporte generado automáticamente por las pruebas de ataque*

Seguridad, usabilidad y eficiencia

El requerimiento de que el sistema sea seguro, usable y eficiente implica equilibrar objetivos que a veces compiten. Las decisiones tomadas fueron las siguientes:

Seguro

El segundo factor es obligatorio para todas las cuentas y no puede desactivarse: la cuenta siempre conserva al menos un método. Las acciones que debilitan la seguridad (eliminar un método, regenerar códigos, cambiar la contraseña) exigen confirmar la contraseña. Los mensajes de error no revelan si un correo está registrado ni si la contraseña era correcta cuando la cuenta está bloqueada.

Usable

- El campo de código acepta pegar el valor completo, muestra seis casillas y envía el formulario automáticamente al escribir el sexto dígito.

- Una cuenta regresiva indica cuándo vence el código y el botón de reenvío se habilita a los 60 segundos.
- Si un método no está disponible, el usuario puede cambiar a otro o usar un código de respaldo desde la misma pantalla.
- Los mensajes explican qué pasó y cómo resolverlo, por ejemplo cuántos intentos quedan o cuántos minutos dura la pausa.
- La interfaz se adapta a teléfonos, respeta el modo oscuro del sistema y la preferencia de movimiento reducido, y es navegable con teclado.

Eficiente

- Validar la contraseña toma cerca de 0.3 segundos. Es un costo intencional de bcrypt: imperceptible para el usuario, pero multiplica el esfuerzo de quien intente descifrar hashes robados.
- La verificación de un código TOTP o por correo toma menos de un milisegundo.
- El sistema funciona con un solo proceso de Node.js y un archivo de base de datos; la instalación completa se reduce a npm install y npm start.

Cuadro de porcentaje de participación

El siguiente cuadro detalla la participación de cada integrante en el proyecto. La suma de los porcentajes corresponde al 100 % del trabajo realizado.

Tabla 7. *Porcentaje de participación de los integrantes*

No.	Integrante	Carné	Actividades realizadas	%
1	Daniel Alejandro González Juárez	0910-22-4469	Desarrollo del servidor y de la lógica de autenticación multifactor; integración de TOTP, correo y SMS; diseño de la interfaz de usuario	50 %
2	Alex Ricardo Pereira	0910-21-14444	Pruebas de resistencia frente a ataques; investigación y marco de referencia (NIST, OWASP, RFC); redacción del informe y presentación	50 %
	Total			**100 %**

Conclusiones

1. La autenticación basada solo en contraseña es insuficiente frente a amenazas actuales como el phishing y la reutilización de credenciales. Con el segundo factor, conocer la contraseña deja de ser suficiente para entrar a una cuenta, como demostró la prueba A2.
2. Pedir un código no basta por sí solo: la protección real proviene de los controles que rodean al segundo factor, como el límite de intentos, la vigencia corta, la prevención de reutilización y la regeneración de la sesión. Sin ellos, un código de seis dígitos podría adivinarse o reutilizarse.
3. La app de autenticación TOTP ofrece el mejor equilibrio entre seguridad y costo, porque no depende de la red móvil ni de un proveedor externo. El correo y el SMS amplían el alcance a más usuarios y funcionan bien como métodos de respaldo.
4. Las pruebas automatizadas permitieron verificar con evidencia objetiva que el sistema resiste 15 de 15 ataques planteados, y además ayudaron a encontrar y corregir debilidades durante el desarrollo.
5. Seguridad y usabilidad no son excluyentes: decisiones de diseño como el envío automático del código, los dispositivos de confianza y los mensajes claros reducen la fricción sin abrir nuevas puertas a los atacantes.

Recomendaciones

1. Antes de un despliegue en producción, servir la aplicación exclusivamente por HTTPS, definir los secretos como variables de entorno y desactivar el modo de demostración.
2. Configurar un proveedor real de correo (SMTP) y de SMS, y monitorear su tasa de entrega.
3. Promover la app de autenticación como método principal y mantener el SMS como respaldo, por el riesgo de duplicado de SIM.
4. Incorporar llaves de seguridad FIDO2/WebAuthn (passkeys), que eliminan el riesgo de phishing en tiempo real.
5. Usar un almacén de sesiones compartido (por ejemplo, Redis) si la aplicación se ejecuta en varios servidores.
6. Integrar la bitácora de auditoría con un sistema de monitoreo (SIEM) y definir alertas para eventos críticos como bloqueos repetidos.
7. Ejecutar las pruebas de ataque de forma periódica y en cada cambio del sistema, como parte de la integración continua.

Referencias

Grassi, P. A., Fenton, J. L., Newton, E. M. et al. (2017, actualizado 2020). NIST Special Publication 800-63B: Digital Identity Guidelines, Authentication and Lifecycle Management. National Institute of Standards and Technology. https://pages.nist.gov/800-63-3/sp800-63b.html

M'Raihi, D., Machani, S., Pei, M. y Rydell, J. (2011). RFC 6238: TOTP: Time-Based One-Time Password Algorithm. IETF. https://www.rfc-editor.org/rfc/rfc6238

M'Raihi, D., Bellare, M., Hoornaert, F., Naccache, D. y Ranen, O. (2005). RFC 4226: HOTP: An HMAC-Based One-Time Password Algorithm. IETF. https://www.rfc-editor.org/rfc/rfc4226

Meyer, L. A., Romero, S., Bertoli, G., Burt, T., Weinert, A. y Lavista Ferres, J. M. (2023). How effective is multifactor authentication at deterring cyberattacks? Microsoft Research. https://www.microsoft.com/research/publication/how-effective-is-multifactor-authentication-at-deterring-cyberattacks/

Verizon. (2025). 2025 Data Breach Investigations Report. Resumen en Help Net Security: https://www.helpnetsecurity.com/2025/04/23/verizon-2025-data-breach-investigations-report-dbir/

OWASP Foundation. (2021). Application Security Verification Standard 4.0.3. https://owasp.org/www-project-application-security-verification-standard/

OWASP Foundation. Multifactor Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html

OWASP Foundation. Authentication Cheat Sheet y Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/

Hunt, T. Have I Been Pwned: Pwned Passwords API (modelo de k-anonimato). https://haveibeenpwned.com/API/v3#PwnedPasswords

Provos, N. y Mazières, D. (1999). A Future-Adaptable Password Scheme (bcrypt). USENIX Annual Technical Conference.

Anexos

Anexo A. Instalación y ejecución

Requisitos: Node.js 18.17 o superior. En la carpeta del proyecto:

```
npm install
npm start              (abre http://localhost:3000)
npm run test:ataques   (ejecuta las 15 pruebas de ataque)
npm run reset          (reinicia la base de datos de demostración)
```

Tabla 8. *Cuentas de demostración*

Rol	Correo	Contraseña	Segundo factor
Usuario	ana@ceiba.test	Quetzal#Lago2026	Correo y SMS
Administrador	admin@ceiba.test	Jade#Volcan2026	Correo

Los códigos de correo y SMS de la demostración se consultan en la bandeja de demostración (/demo/bandeja). Para la app de autenticación, se crea una cuenta nueva y se escanea el código QR con Google Authenticator, Microsoft Authenticator o Authy.

Anexo B. Capturas adicionales

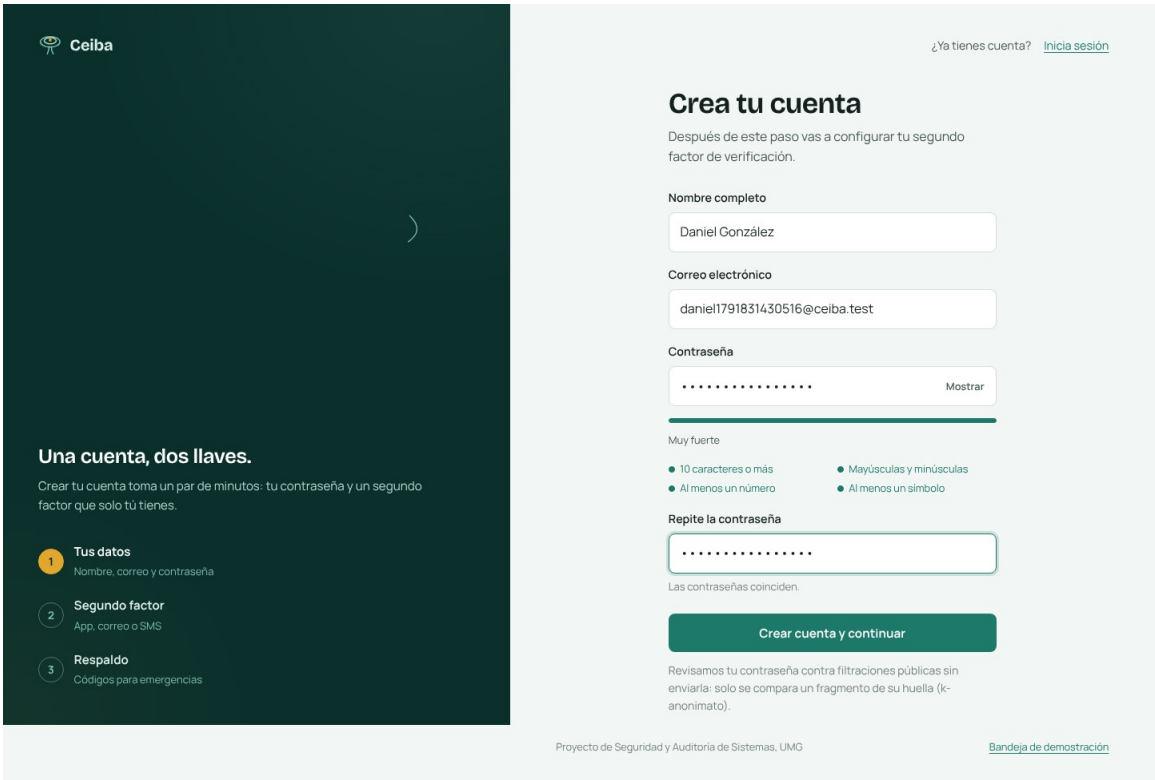


Figura 8. *Registro con medidor de fortaleza de contraseña*



Figura 9. *Elección del segundo factor al crear la cuenta*

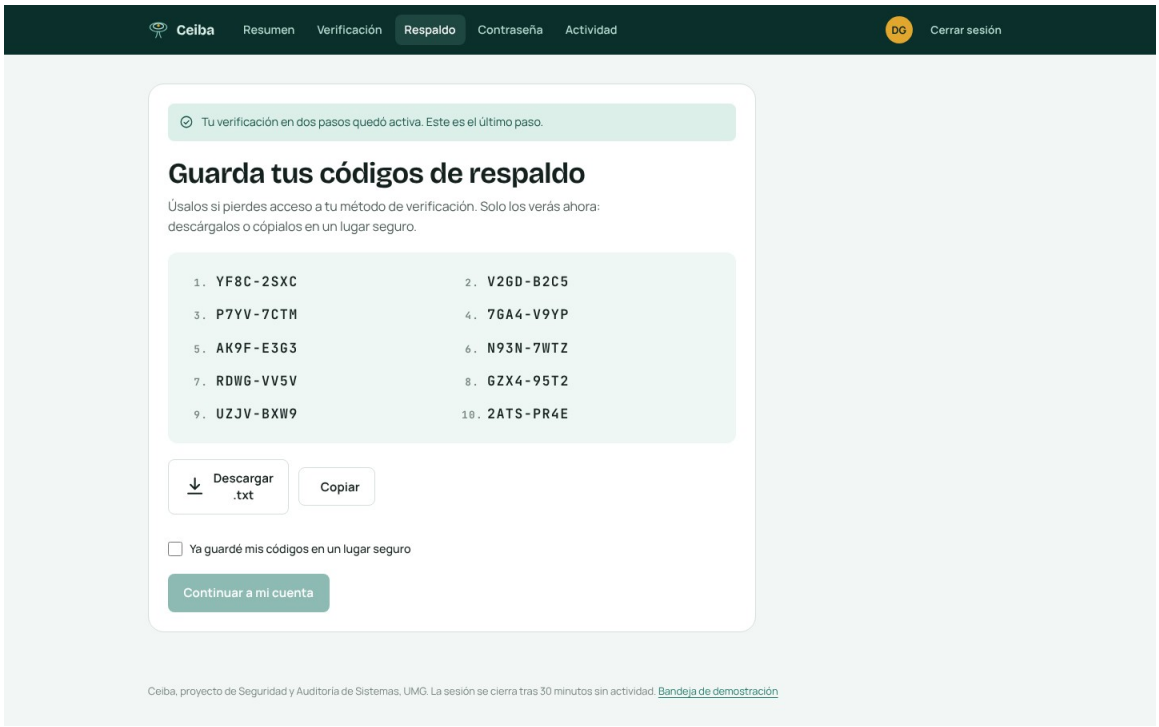


Figura 10. *Códigos de respaldo mostrados una sola vez*

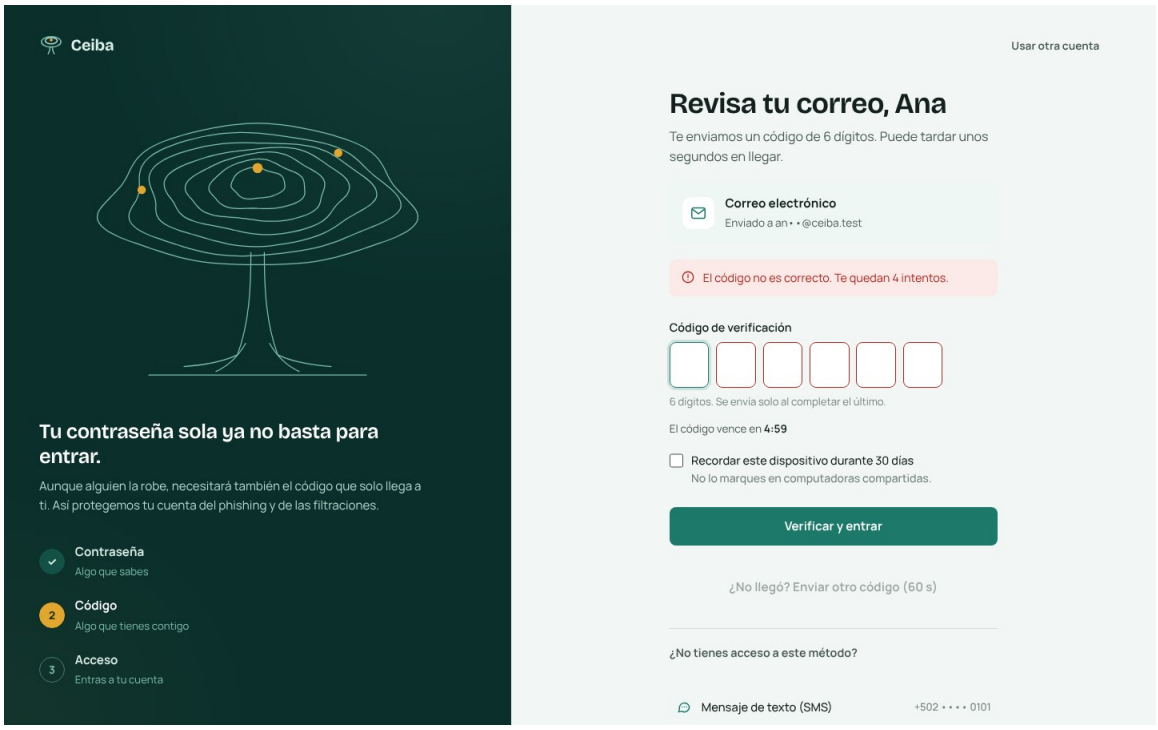


Figura 11. *Código incorrecto: se informa cuántos intentos quedan*

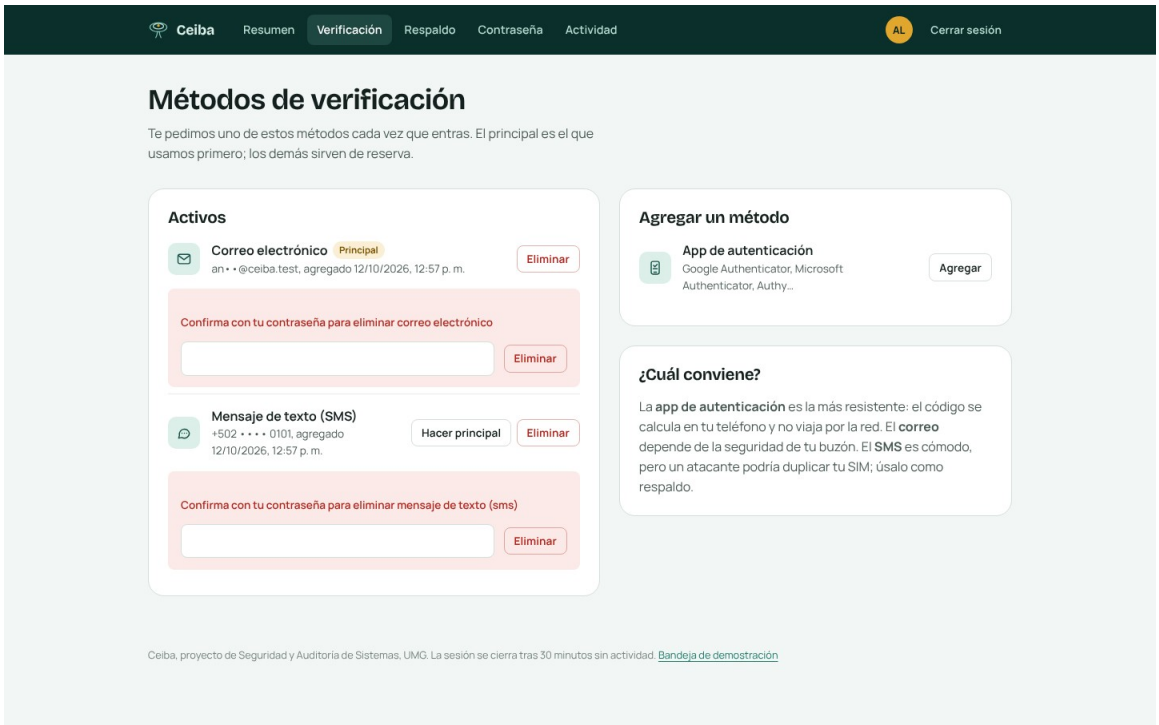


Figura 12. *Administración de métodos de verificación*

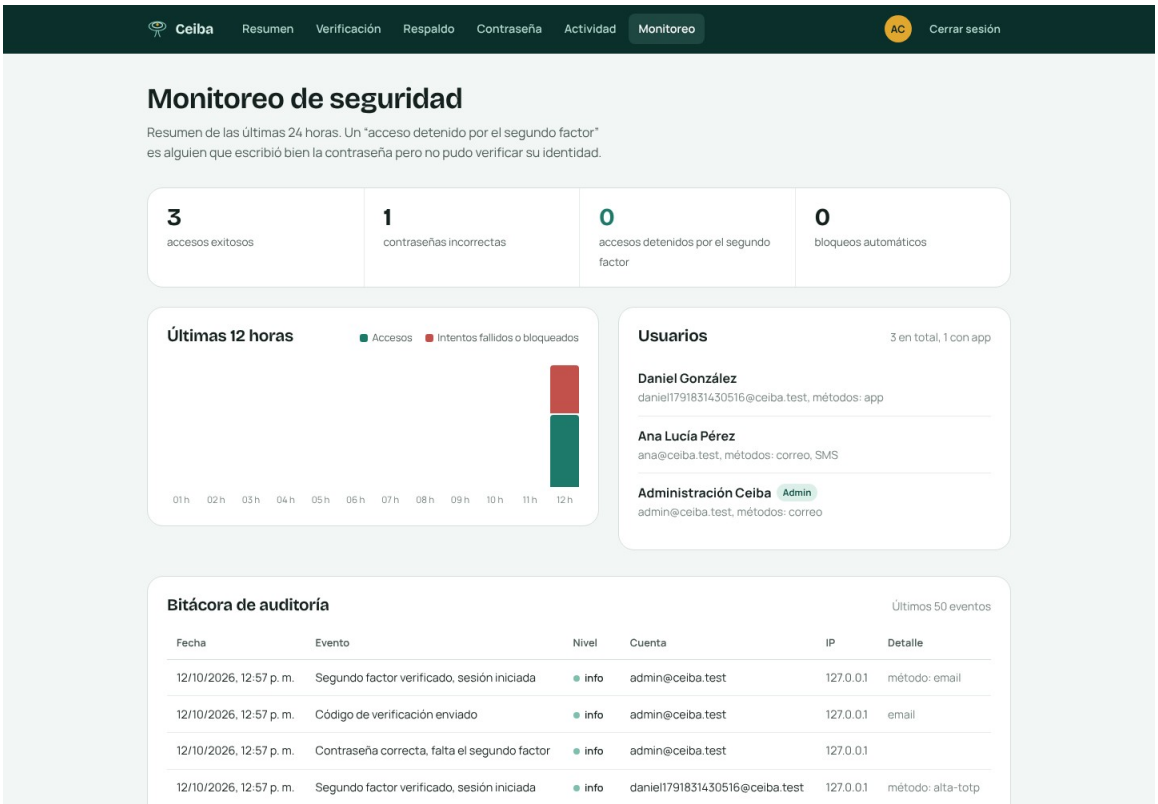


Figura 13. *Panel de monitoreo de seguridad del administrador*

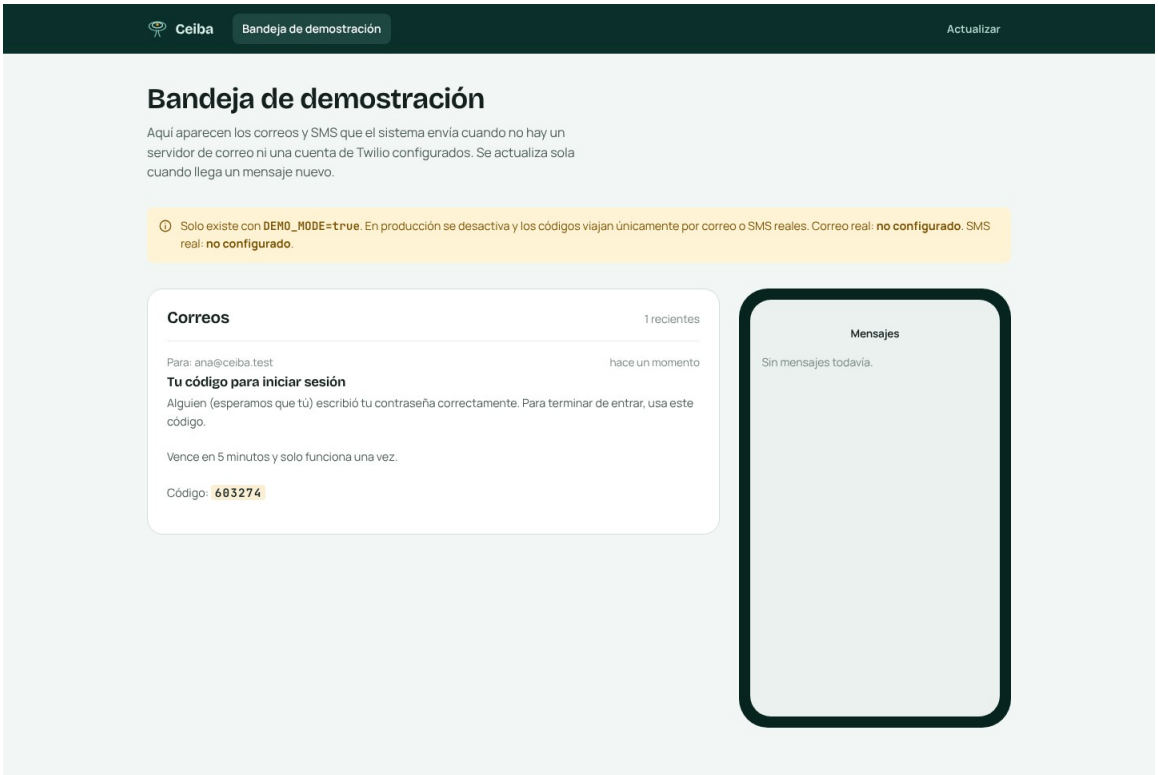


Figura 14. *Bandeja de demostración con correos y SMS simulados*